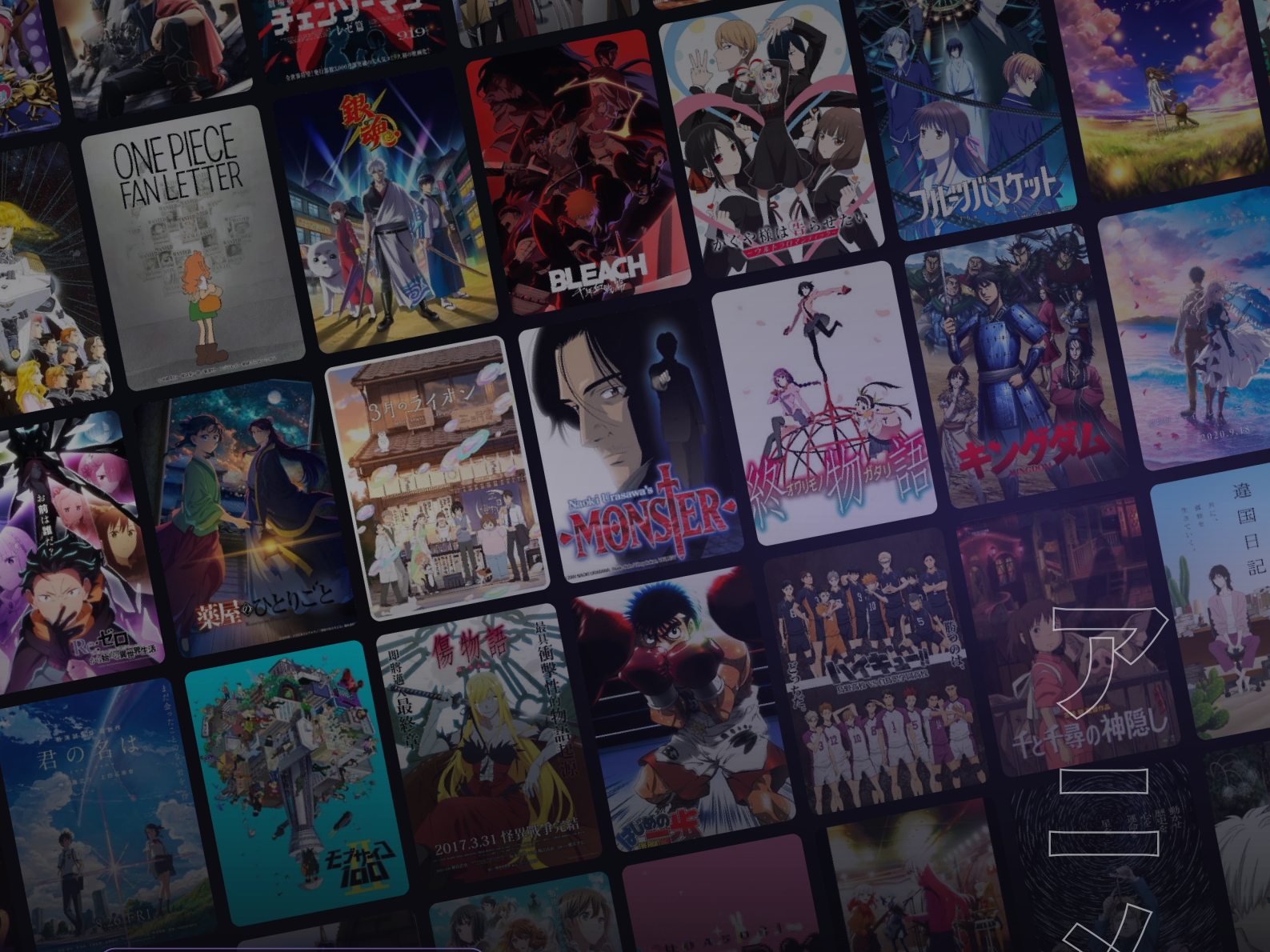


ĐỀ XUẤT CẢI TIẾN · 09/11/2026

AnimeTracking khiến người xem phải „Ồ!“

Đọc toàn bộ dự án, đo đạc dữ liệu thật, đào sâu Jikan API v4 cùng giới hạn của nó — rồi biến tất cả thành 12 ý tưởng chạy được trên một site **hoàn toàn static** ở GitHub Pages.

01 Hiện trạng & 10 lỗi đo được

02 Jikan API: khả năng & giới hạn

03 Kiến trúc static 3 tầng

04 12 ý tưởng „Ồ!“ + lộ trình

Repo: github.com/TriNGUYEN0/AnimeTracking

Live: tringuyen0.github.io/AnimeTracking

00 · TÓM TẮT

Một trang để nắm hết

Dự án có nền tốt (Angular 20 + signals, dashboard lọc chéo, Omikuji và màn hình chờ bát ramen rất có cá tính). Nhưng hiện nó là **một file JSON 19 MB đóng băng từ 13/05/2026 + ba trang biểu đồ**: không có gì „sống“, không có gì để chơi hay chia sẻ, và dù tên là *Tracking* nhưng chưa theo dõi được gì.

5,05 MB
gzip phải tải mỗi lượt vào (19 MB JSON, parse riêng ở 4 component). Index tối ưu: **651 KB**.

5,14 ≠ 6,44
„Average Score“ trên Dashboard bị sai: 5.060 anime chưa có điểm bị tính là 0.

76,6 %
anime không có năm → biểu đồ theo năm & bộ lọc thập niên bỏ qua 19.147 mục.

1 / 14
khả năng Omikuji ở trang chủ bốc ra một tựa Hentai/Erotica (1.719 mục, 6,9%).

BA BƯỚC LỚN

Bước 1 · Nền móng

Dữ liệu tự cập nhật

Pipeline GitHub Actions gọi Jikan theo lịch, tôn trọng limit, xuất dữ liệu tách mảnh (index 651 KB + mảnh tải khi cần). Sửa 4 lỗi số liệu, lọc SFW, bỏ trùng.

Bước 2 · Lớp sống

Gọi Jikan từ trình duyệt

Dữ liệu nóng (đang chiếu, lịch phát sóng, nhân vật, trailer) lấy live qua một hàng đợi ≤ 3 req/s · ≤ 60 req/phút, cache 24 h, tự lùi về snapshot khi API lỗi.

Bước 3 · „Ồ!“

Thứ để chơi & khoe

Anime Galaxy, Animedle (đoán anime mỗi ngày), Radar phát sóng đếm ngược, MAL Wrapped, Watch Order, Seiyuu Six Degrees… 12 ý tưởng xếp theo độ wow / công sức.

NẾU CHỈ CÓ MỘT TUẦN

1. **Sửa 4 lỗi số liệu + lọc SFW** — nửa ngày, tác động lớn nhất.
2. **Một DataStore dùng chung + lazy route** — hết tải 19 MB nhiều lần, bundle nhẹ hơn.
3. **Pipeline Actions + index 651 KB** — dữ liệu luôn mới, ngày cập nhật hiển thị tự động.
4. **Radar phát sóng** có đồng hồ đếm ngược theo giờ của người xem.
5. **Animedle** — trò chơi mỗi ngày, chia sẻ lưới emoji: lý do để quay lại.

BA ĐIỀU VỀ JIKAN CẦN NHỚ

- **60 request/phút** là trần cứng (3/s chỉ là nhịp), không giới hạn theo ngày, MAL có thể chặn ngược khiến bạn nhận 429 dù chưa vượt.
- Không có API key → limit tính theo **IP của người gọi**: trên site static, mỗi khách truy cập có quota riêng.
- `/users/{name}/animelist` **đã ngừng từ 01/05/2022**. Lúc viết tài liệu này, `api.jikan.moe` còn timeout liên tục → luôn cần snapshot dự phòng.

HÔM NAY → SAU LỘ TRÌNH

	HÔM NAY	SAU P1 (NỀN MÓNG)	SAU P4 (ĐẦY ĐỦ)
Dữ liệu tải lần đầu	5,05 MB gzip	**651 KB**	651 KB + mảnh tải khi cần
Độ tươi dữ liệu	đóng băng từ 13/05/2026	catalogue ≤ 7 ngày	airing ≤ 24 h, chi tiết live
Lỗi dữ liệu nghiêm trọng	4 (điểm TB, năm, 18+, trùng)	**0**	0
Lý do để quay lại	không có	—	Animedle, Radar, Tracker, Wrapped

01 · HIỆN TRẠNG

Dự án đang có gì

Angular 20 standalone + signals, hash routing, deploy bằng `angular-cli-ghpages`. Toàn bộ dữ liệu đến từ một file tĩnh do script Python crawl thủ công.

/ · Home

- Hero = anime điểm cao nhất (Frieren) + đoạn giới thiệu
- 6 nút „mood“ lọc theo genre, carousel cố định 5 thẻ
- Omikuji 御神籤: lắc hộp → bốc anime ngẫu nhiên
- Màn hình chờ bát ramen bằng CSS thuần

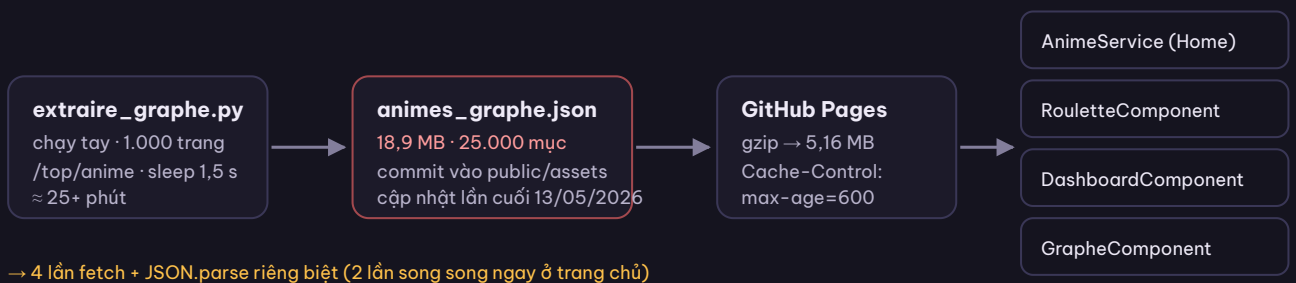
/dashboard

- 4 KPI + 5 biểu đồ Chart.js (line, bar, polar, doughnut)
- Lọc chéo: click studio/genre → mọi biểu đồ cập nhật (computed signals)
- Lọc thập niên, xuất PDF (html2canvas + jsPDF)

/graphe

- ECharts force graph: node = poster, cạnh = cùng studio *hoặc* cùng năm
- Lọc studio / năm / genre, tìm trong tiêu đề + synopsis
- Panel chi tiết kính mờ khi click node

LUỒNG DỮ LIỆU HIỆN TẠI



→ 4 lần fetch + JSON.parse riêng biệt (2 lần song song ngay ở trang chủ)

NÊN GIỮ

- Omikuji + bát ramen: chất „anime“ riêng, nâng cấp chứ đừng bỏ.
- `computed()` cho lọc chéo dashboard: gọn, đúng hướng Angular hiện đại.
- Hash routing (`withHashLocation`) hợp với GitHub Pages — không cần trick 404.html.
- View Transitions đã bật — nền tảng sẵn cho hiệu ứng chuyển cảnh poster.

ĐANG THIẾU

- Không có trang chi tiết anime, không có tìm kiếm toàn cục, không deep link.
- Dữ liệu đóng băng; ngày cập nhật viết tay trong HTML.
- Không responsive (carousel 5 thẻ cố định, graph không cho cảm ứng).
- UI trộn Pháp/Anh, code trộn Pháp/Anh/Việt; `backend/` Flask không còn dùng.
- Không có tính năng **tracking** nào dù đó là tên dự án.

DATASET HIỆN TẠI TRONG VÀI CON SỐ

25.000 mục (24.852 id duy nhất)

1.340 studio khác nhau

21 genre (chưa có themes, demographics)

5.060 mục không có điểm

8.077 mục không có studio

1961–2027 khoảng năm (chỉ 23% có năm)

10 vấn đề, có số đo & dòng code

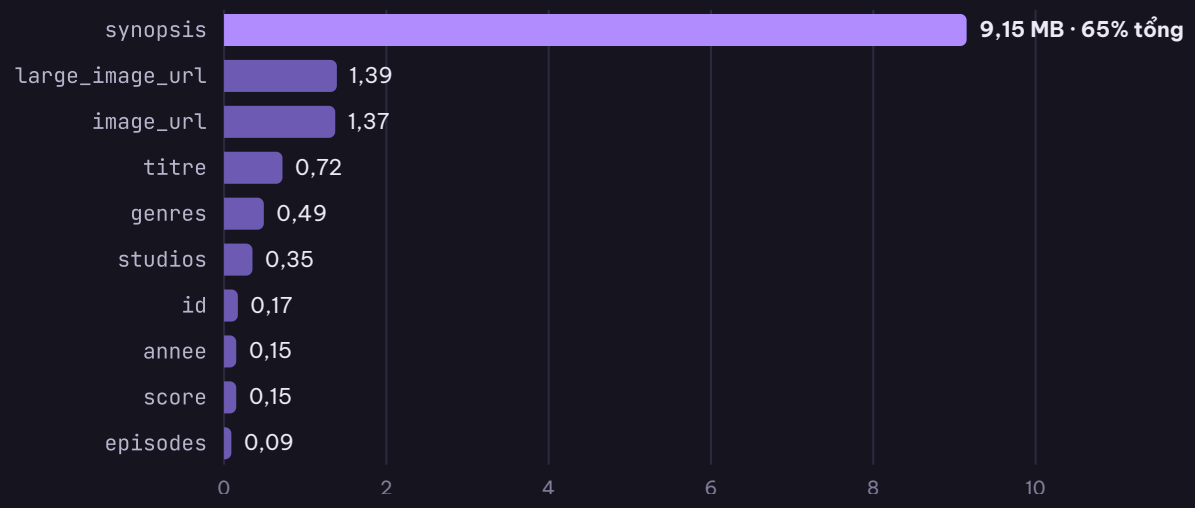
Đo trực tiếp trên `front/public/assets/animes_graphe.json` và bản live ngày 09/11/2026.

MỨC	VẤN ĐỀ	BẰNG CHỨNG	CÁCH SỬA
NGHIÊM TRỌNG	**Điểm trung bình sai**: hiển thị 5,14; đúng là 6,44	`dashboard.ts:114`, `analytics.service.ts:57` chia tổng cho *mọi* anime, gồm 5.060 mục không có điểm (Jikan trả `score` rỗng/0 khi chưa xác định).	Chỉ chia cho số anime có điểm; hiển thị kèm „n = …“.
NGHIÊM TRỌNG	**76,6% anime không có năm**	`extraire_graphe.py:23` lấy `year` = năm của *mùa phát sóng TV*, null với movie/OVA/ONA. Ví dụ: *Kimi no Na wa.*, *Gintama: The Final*, cả 2 movie Gintama.	Dùng `aired.prop.from.year` (fallback `aired.from`). Code Flask cũ đã làm đúng việc này.
NGHIÊM TRỌNG	**Nội dung 18+ trên site công khai**	1.719 mục Hentai/Erotica. `roulette.ts:51` bốc ngẫu nhiên trên toàn bộ danh sách → 1/14 lượt; Graph và Dashboard cũng hiển thị.	Crawl với `sfw=true`; loại `rating=rx`; tùy chọn ẩn Ecchi.
CAO	**148 mục trùng**	Crawl 25 phút theo thứ hạng trong khi bảng xếp hạng thay đổi → trang bị trượt, mục lặp lại.	Bỏ trùng theo `mal_id`; crawl `/anime?order_by=mal_id` (thứ tự ổn định).
CAO	**19 MB tải & parse 4 lần**	`anime.service.ts:24`, `roulette.ts:35`, `dashboard.ts:529`, `graphe.ts:52` — mỗi nơi tự `http.get`. Synopsis chiếm 9,15 MB.	Một `DataStore` root + `shareReplay`; index 651 KB; synopsis tải theo mảnh.
CAO	**Bundle chính 1,7 MB** (558 KB gzip)	`graphe.ts:5` `import * as echarts`; `app.config.ts:14` đăng ký mọi loại chart; mọi route tải sẵn (eager).	`loadComponent()` từng route; `echarts/core` modular; dùng 1 thư viện chart.
TRUNG BÌNH	**Graph O(n²), cạnh ít ý nghĩa**	`graphe.ts:166-179`: 1.000 node → 499.500 phép so sánh và 17.063 cạnh, đa số chỉ vì „cùng năm“.	Cạnh = recommendations / relations / seiyuu, tính sẵn trong pipeline.
TRUNG BÌNH	`provideRouter` **gọi 2 lần**	`app.config.ts:11` và `:13`, mỗi lần một feature → dễ mất một trong hai.	`provideRouter(routes, withHashLocation(), withViewTransitions())`.
THẤP	**Nội dung cứng, typo**	`home.html:40` „API Jinkan“; `:42` ngày cập nhật viết tay; `anime.service.ts:36` rank = vị trí trong file.	Đọc `meta.json` do pipeline sinh; dùng `rank` của MAL.
THẤP	**Ảnh nặng & vòng chuyển hướng**	URL `myanimelist.net/images/…` trả 301 sang CDN; ảnh .jpg 56,7 KB trong khi .webp chỉ 15,7 KB (Frieren).	Lưu path ngắn, dựng URL `cdn.myanimelist.net/…webp`; `loading="lazy"`.

19 MB đi đâu — và cách giảm còn 651 KB

Kích thước từng trường khi serialize toàn bộ 25.000 mục. Hai phần ba là synopsis — thứ chỉ cần khi người dùng mở chi tiết một anime.

DUNG LƯỢNG THEO TRƯỜNG (MB, JSON CHƯA NÉN)



18,9 MB
Hiện tại · 25.000 mục · 5,05 MB gzip · parse 4 lần

651 KB
Index gzip cho 23.137 mục SFW, không trùng (1,61 MB raw) · **−87%**

134 KB
Top 5.000 gzip — đủ cho Galaxy & Animedle

~13 KB
mỗi mảnh synopsis (256 mảnh theo `id % 256`), chỉ tải khi mở chi tiết

KỸ THUẬT ĐÃ THỬ TRÊN DỮ LIỆU THẬT

- Mảng thay cho object, khóa ngắn: `[id, title, year, score, eps, [g], [s], img]`.
- Genre/studio → bảng tra + chỉ số nguyên (1.340 studio chỉ lưu tên 1 lần).
- Ảnh chỉ lưu `"1015/138006"`, dựng lại `https://cdn.myanimelist.net/images/anime/{p}.webp` (thêm `l` cho bản lớn).
- Bỏ trùng theo `mal_id`, lọc SFW trước khi ghi.
- Parse trong Web Worker để không giật giao diện khi tải.

TRƯỜNG ĐANG BỊ BỎ PHÍ

Script hiện chỉ giữ 10 trường. Jikan trả về nhiều thứ đáng giá hơn mà gần như không tốn thêm request:

`members` `favorites` `rank` `popularity` `scored_by` `type` `source` `rating` `season` `status` `themes` `demographics` `title_english` `title_japanese` `trailer.youtube_id` `broadcast` `aired.from/to` `duration`

`themes` (Isekai, Time Travel, Gore, Iyashikei…) và `demographics` (Shounen, Seinen…) chính là nguyên liệu cho Galaxy, gợi ý theo mood và persona của Wrapped.

02 · JIKAN API V4

Hiểu đúng Jikan trước khi dùng

Jikan là API **không chính thức**: nó scrape MyAnimeList rồi trả JSON. Chỉ có `GET`, chỉ đọc, không cần key, không đăng nhập — 100 endpoint chia thành anime, manga, characters, people, producers, seasons, schedules, top, random, users, watch, reviews, recommendations, clubs, genres, magazines.

RATE LIMIT (TÀI LIỆU CHÍNH THỨC)

Mỗi giây	**3** request
Mỗi phút	**60** request — trần thực sự
Mỗi ngày	**Không giới hạn**
Phía MAL	MAL có thể chặn chính Jikan → bạn nhận `429` dù chưa vượt

Maintainer giải thích trong issue #461: được 60 request/phút, chia đều mỗi giây hay dồn vài request một giây là tùy bạn — trần là 60/phút. Các thư viện client thường giãn ~350 ms/request.

CACHE & XÁC THỰC CACHE

- Mọi request được Jikan cache **24 giờ** → gọi lại trong ngày chỉ nhận đúng dữ liệu cũ; cache phía bạn 24 h là vừa khớp.
- Header: `Expires`, `Last-Modified`, `ETag` (MD5 của body).
- Gửi `If-None-Match: <etag>` → `304 Not Modified` nếu không đổi: không phải tải lại/ghi lại.
- `X-Request-Fingerprint` chỉ có ở resource đơn (`/anime/1`, `/anime/1/relations`), không có ở truy vấn (`/anime`, `/top/anime`).

MÃ LỖI

`304`	Không đổi (khi gửi ETag)
`400`	Tham số sai
`404`	Không có hoặc MAL trả 404
`429`	Bị rate-limit (bởi Jikan hoặc MAL)
`500`	Lỗi parse/upstream, kèm `report_url`
`503`	Bảo trì

THAM SỐ HỮU ÍCH

- `sfw=true` lọc nội dung người lớn; `rating=g|pg|pg13|r17|r|rx`.
- `order_by`: mal_id, score, scored_by, rank, popularity, members, favorites, start_date…
- `/top/anime?filter=airing|upcoming|bypopularity|favorite`.
- `/schedules?filter=monday…sunday`, `kids`, `sfw`.
- `start_date` / `end_date`, `genres`, `genres_exclude`, `producers`, `letter`.
- Phân trang: 25 mục/trang, kèm `pagination.has_next_page`.

Đã ngừng: `/users/{name}/animelist` và `/mangalist` từ 01/05/2022. Vẫn còn: `/full`, `/statistics`, `/favorites`, `/history`, `/userupdates`.

Bẫy dữ liệu: `score` = 0 khi chưa xác định; ngày giờ là ISO 8601 UTC; `broadcast` có `timezone` (thường Asia/Tokyo).

Thực tế 09/11/2026: `api.jikan.moe` timeout liên tục khi viết tài liệu này, trong khi `jikan.moe` vẫn chạy. Giao diện không bao giờ được phụ thuộc 100% vào API live.

Endpoint nào mở khóa tính năng nào

Chi phí tính theo số request. „Pipeline“ = chạy trong GitHub Actions; „Live“ = gọi từ trình duyệt người xem.

ENDPOINT	TRẢ VỀ	MỞ KHÓA	CHI PHÍ
`/seasons/now` · `/seasons/upcoming`	Anime mùa này / sắp chiếu, có trailer & broadcast	Hero điện ảnh, „Mùa này“	3–6 req/ngày · Pipeline
`/schedules?filter={day}`	Lịch phát theo thứ, giờ JST	Radar phát sóng, đếm ngược	7–14 req/ngày · Pipeline
`/top/anime?filter=…`	4 bảng xếp hạng: airing, upcoming, bypopularity, favorite	Bảng xếp hạng động, Higher-or-Lower	1 req / 25 mục
`/anime?q=&genres=&order_by=&sfw`	Tìm kiếm + lọc phía server	Crawl catalogue, tìm kiếm live	1 req / 25 mục
`/anime/{id}/full`	+ relations, theme (OP/ED), streaming, external, trailer	Trang chi tiết, Watch Order, Jukebox	1 req · Live/Pipeline
`/anime/{id}/statistics`	watching / completed / on_hold / dropped / plan_to_watch + phân phối điểm 1–10	Score DNA, tỉ lệ drop	1 req · Pipeline
`/anime/{id}/characters`	Nhân vật + vai trò + seiyuu theo ngôn ngữ	Seiyuu Six Degrees	1 req · Pipeline
`/anime/{id}/recommendations`	Cặp gợi ý do người dùng MAL bình chọn	Cạnh của Galaxy / Graph	1 req · Pipeline
`/anime/{id}/episodes`	Tiêu đề, ngày chiếu, **điểm từng tập 1–5**, cờ **filler/recap**	Heatmap tập, lộ trình bỏ filler	1 req / 100 tập · Live
`/anime/{id}/videos` · `/pictures`	Promo, MV, tập có video; ảnh	Jukebox, gallery	1 req · Live
`/characters/{id}/full` · `/people/{id}/full`	Tiểu sử, vai diễn, ảnh	Trang nhân vật & seiyuu	1 req · Live
`/top/characters`	Nhân vật được yêu thích nhất	Bracket „Best character“	1 req / 25
`/producers/{id}/full`	Thông tin studio, liên kết	Trang studio	1 req · Live
`/watch/episodes` · `/watch/promos`	Tập & promo mới nhất (+ bản popular)	Feed „Vừa ra lò“	1–2 req/ngày
`/users/{u}/full` · `/favorites` · `/history`	Hồ sơ, thống kê, yêu thích, lịch sử gần đây	MAL Wrapped	2–3 req / người · Live
`/genres/anime`	Danh sách genre/theme + id + số lượng	Bộ lọc, mood	1 req / tuần

02 · NGÂN SÁCH REQUEST

Làm toán với 60 request/phút

Pipeline chạy ở nhịp **1 request / 1,1 s ≈ 55/phút**: chừa 5 lượt mỗi phút cho retry. Runner của GitHub Actions có thể dùng chung IP với người khác, nên càng không nên chạy sát trần.

MỘT PHÚT CỦA CRAWLER



● 55 request, cách nhau 1,1 s · ~5 lượt trống còn lại cho retry khi gặp 429/5xx

CÔNG VIỆC	REQUEST	THỜI GIAN @55/PHÚT	TẦN SUẤT
Catalogue SFW qua `/anime?order_by=mal_id&sfw=true`	≈ 950–1.150	≈ 17–21 phút	Mỗi tuần
Mùa này + lịch phát sóng (`/seasons/now` , `/schedules` × 7)	≈ 15–20	< 1 phút	Mỗi ngày (hoặc 2 lần)
Làm giàu top 5.000: `/full` + `/statistics` + `/characters` + `/recommendations`	20.000	**≈ 6 giờ 4 phút**	Vượt trần 6 h/job → chia 7 đêm
→ **Xoay vòng 1/7 mỗi đêm** (`mal_id % 7 == thứ`)	≈ 2.860	≈ 52 phút	Mỗi đêm, mới hoàn toàn sau 1 tuần
Episodes cho ~300 bộ dài / đang chiếu	≈ 600	≈ 11 phút	Mỗi tuần

PHÍA TRÌNH DUYỆT

- Mở chi tiết một anime = 3 request (`/full` , `/characters` , `/videos`) → giãn 400 ms ≈ 1,2 s; lần mở sau: **0 request** nhờ IndexedDB.
- Không có key → limit tính theo IP: 1.000 khách cùng lúc vẫn là 1.000 quota riêng. Nhưng hãy lịch sự: cache 24 h, gộp request trùng, ưu tiên snapshot.
- Đồng bộ nhiều tab bằng `BroadcastChannel` hoặc Web Locks để 2 tab không cùng „đốt“ quota.

QUY TẮC VÀNG

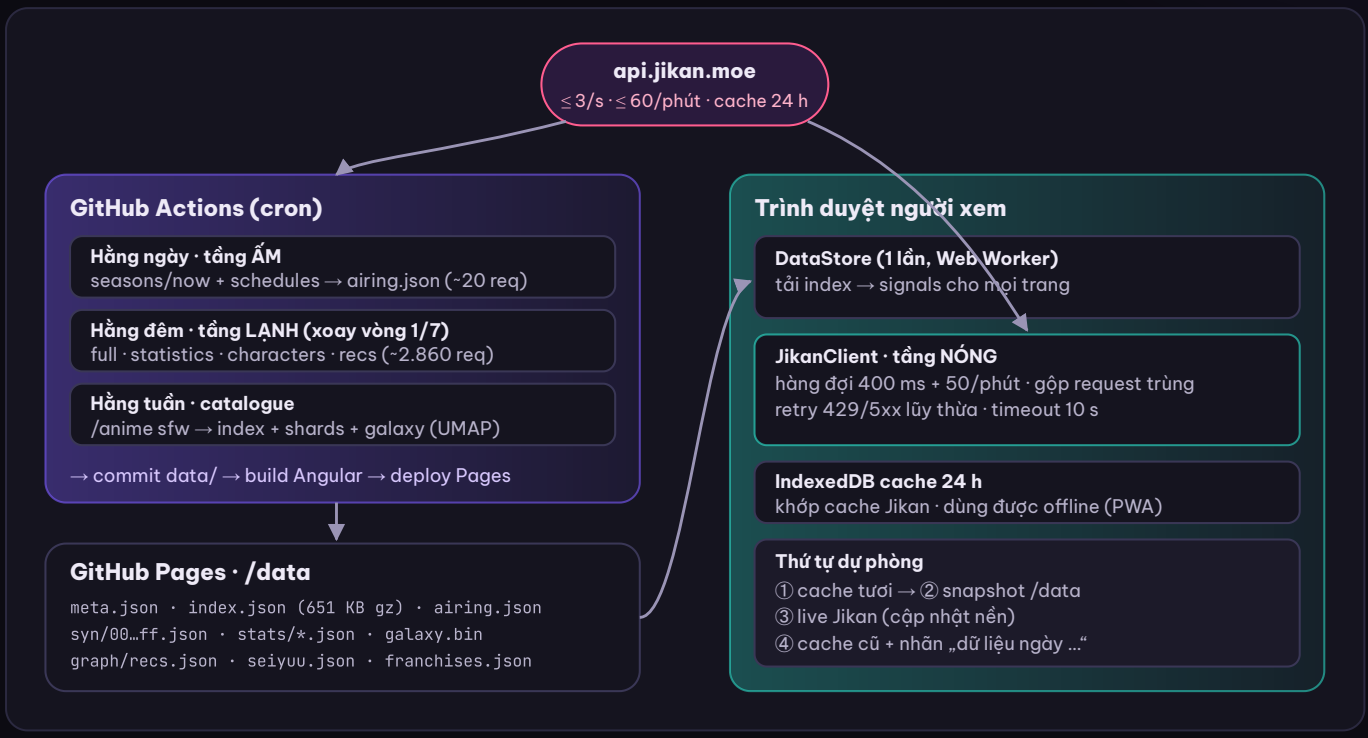
1. **Snapshot trước, live sau** (stale-while-revalidate): hiện ngay dữ liệu tĩnh, cập nhật khi API trả lời.
2. **429/5xx → lùi lũy thừa** 1 → 2 → 4 → 8 s (+ jitter), tôn trọng `Retry-After` nếu có.
3. **Timeout 10 s** rồi rơi về dữ liệu cũ; không bao giờ spinner vô hạn.
4. **ETag + If-None-Match** trong pipeline: mục không đổi thì bỏ qua ghi file và commit.
5. Đặt `User-Agent` rõ ràng cho crawler (tên dự án + link repo).

CRAWLER CŨ VS MỚI

	`testapi/extraire_graphe.py` (HIỆN TẠI)	`pipeline/crawl.py` (ĐỀ XUẤT)
Cách chạy	Chạy tay trên máy cá nhân	GitHub Actions theo lịch + nút chạy thủ công
Nguồn & thứ tự	`/top/anime` theo rank → trượt trang, 148 mục trùng	`/anime?order_by=mal_id&sfw=true` → ổn định, bỏ trùng theo id
Nhịp & lỗi	`sleep(1,5)` cố định; trang lỗi bị bỏ qua, không retry	1,1 s/request, retry lũy thừa cho 429/5xx, ETag/304
Trường dữ liệu	10 trường, năm sai cho 76,6% mục	~25 trường (members, themes, trailer, broadcast…), năm từ `aired`
Đầu ra	1 file 18,9 MB	index 651 KB gzip + mảnh + `meta.json` có ngày cập nhật

Static nhưng vẫn sống: 3 tầng dữ liệu

GitHub Pages chỉ phục vụ file tĩnh, nên „backend“ chính là GitHub Actions chạy theo lịch, còn trình duyệt tự gọi Jikan cho phần cần tươi.



GIỚI HẠN GITHUB CẦN BIẾT

- Site Pages ≤ **1 GB**; băng thông mềm **100 GB/tháng**; deploy timeout **10 phút**.
- Soft limit 10 build/giờ **không áp dụng** khi deploy bằng workflow Actions riêng.
- Job trên runner GitHub tối đa **6 giờ** → lý do phải chia nhỏ việc làm giàu dữ liệu.
- Repo public: workflow `schedule` **tự tắt sau 60 ngày** không có hoạt động → để workflow commit dữ liệu (repo luôn có hoạt động) và vẫn nên liếc tab Actions.
- Đã đo: Pages tự gzip JSON (19 MB → 5,16 MB) và gửi `Cache-Control: max-age=600` → đặt tên file có hash/phiên bản để chủ động làm mới.

VÌ SAO KHÔNG CẦN BACKEND

- Jikan bật CORS (được dùng rộng rãi từ trình duyệt) → gọi trực tiếp, không cần proxy. Chưa kiểm được header hôm nay vì API timeout; nên xác nhận lại.
- CDN ảnh MAL trả `Access-Control-Allow-Origin: *` (đã kiểm) → canvas đọc được pixel: lấy màu chủ đạo, xuất ảnh chia sẻ PNG ngay phía client.
- Mọi thứ nặng (UMAP, đồ thị seiyuu, thống kê) tính sẵn bằng Python trong Actions — trình duyệt chỉ việc vẽ.
- Thư mục `backend/` Flask có thể chuyển sang `pipeline/` : tái dùng logic pandas cho bước tiền xử lý.

Xếp theo độ „Ồ!“ và công sức

Đánh giá chủ quan trên thang 1–10. Góc trên-trái là chỗ nên bắt đầu: hiệu ứng lớn, làm nhanh. Góc trên-phải là các dự án chủ lực làm nên tên tuổi của site.



LÀM NGAY

③ Radar phát sóng
② Animedle
⑨ Hero + OP Jukebox
⑦ Score DNA

CHỦ LỰC

① Anime Galaxy
④ MAL Wrapped
⑥ Seiyuu Six Degrees
⑧ Heatmap & Filler
⑤ Watch Order

ĐÁNH BÓNG

⑩ Bar chart race
⑫ ⌘K + Mood
Rẻ, làm xen kẽ khi cần đổi gió.

NỀN CHO MỌI THỨ

⑪ Tracker cá nhân
Không „wow“ nhất, nhưng biến khách vãng lai thành người quay lại — và đúng với cái tên.

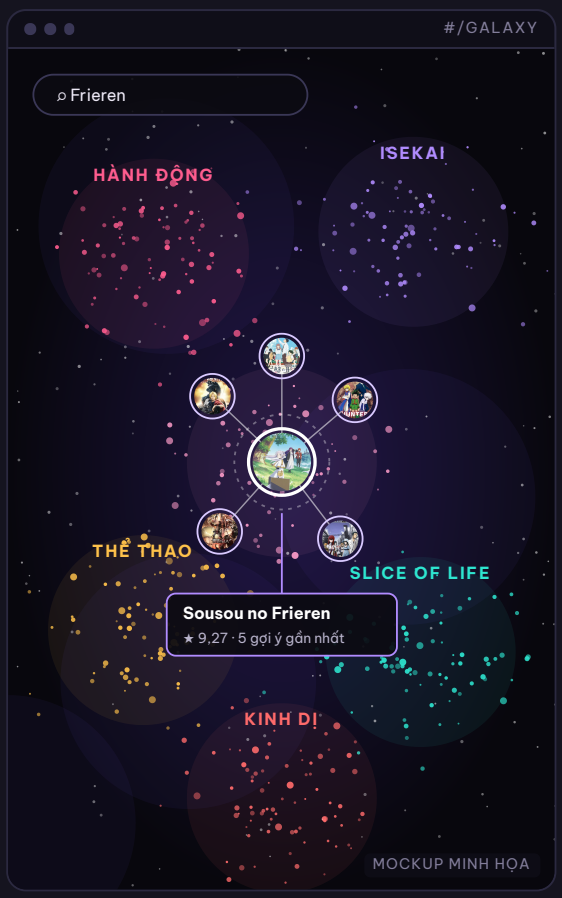
Ý TƯỞNG 01 · CHỦ LỰC

Anime Galaxy 星雲

Ồ! Mở trang là thấy 5.000 anime trôi như dải ngân hà. Gõ „Frieren“ → camera bay xuyên thiên hà tới đúng ngôi sao đó, các đường chòm sao nối sang những bộ „nếu thích cái này“.

- Pipeline dựng vector cho mỗi anime: genres + themes + demographics + đồ thị recommendations → **UMAP** 2D bằng Python (`umap-learn`).
- Xuất `galaxy.bin` (Float32: x, y + id): ~60 KB cho 5.000 điểm.
- Render WebGL (PixiJS / regl / deck.gl) với LOD: nhìn xa = chấm sáng tô theo genre chính, zoom gần = poster thật, click = panel chi tiết live.
- Thay thế hẳn graph hiện tại (O(n²), cạnh „cùng năm“).

`/anime/{id}/recommendations` `5.000 req ≈ 91 phút, xoay vòng` `Khó · 8/10`



Ý TƯỞNG 02 · LÀM NGAY

Animedle — đoán anime mỗi ngày

Ồ! Cả thế giới nhận *cùng một* câu đố mỗi ngày. Đoán sai thì nhận gợi ý kiểu Wordle: năm ↑↓, studio ✓, genre trùng một phần, điểm ↑↓. Xong thì chia sẻ lưới emoji lên mạng.

- Không cần server: `seed = hash(ngày UTC)` → chọn trong top 1.000 theo `members` (đủ nổi tiếng để công bằng).
- 5 chế độ: Classic (so thuộc tính) · Poster (blur 40 px → 0 sau mỗi lần sai) · Emoji (synopsis → emoji) · OP (tên bài mở đầu từ `theme.openings`) · Bóng nhân vật.
- Ô đoán có autocomplete từ index; streak & thống kê lưu localStorage.
- Nút „Copy kết quả“ tạo chuỗi 🟩🟧⬛ + link — vòng lặp lan truyền miễn phí.

`0 request khi chơi` `dùng snapshot` `Dễ · 4/10`



Ý TƯỞNG 03 · LÀM NGAY

Radar phát sóng

Ồ! „Tập mới lên sóng sau **02:13:45**“ — đồng hồ đếm ngược thật, theo giờ địa phương của chính người xem; dải „ĐANG CHIẾU“ chạy chữ như bảng ga tàu Tokyo.

- Pipeline hằng ngày: `/schedules` × 7 ngày + `/seasons/now` → `airing.json`.
- `broadcast {day, time, timezone: "Asia/Tokyo"}` → đổi sang giờ người xem bằng `Intl.DateTimeFormat` (hoặc Temporal khi trình duyệt hỗ trợ).
- Lưới tuần 7 cột, nút „Theo dõi“ (localStorage), xuất `.ics` sinh phía client để thêm vào lịch.
- Lưu ý: lịch MAL là giờ chiếu TV Nhật; bản streaming quốc tế có thể lệch → ghi chú rõ trên UI.

`/schedules` `/seasons/now` `~20 req/ngày` `Dễ · 3/10`



Ý TƯỞNG 04 · CHỦ LỰC

MAL Wrapped

Ồ! Nhập username MyAnimeList → chuỗi slide kiểu Spotify Wrapped: „Bạn đã xem **143,2 ngày** anime“, „Bạn chấm khắt hơn 78% cộng đồng“, nhân vật yêu thích, „nhân cách anime“ — rồi tải ảnh để khoe.

- `/users/{u}/full` (kèm statistics) + `/favorites` + `/history`: 2–3 request live qua hàng đợi.
- Persona từ genre/theme của favorites (vd. „Pháp sư Shounen“, „Người gác đền Iyashikei“).
- Xuất PNG bằng canvas: CDN ảnh MAL đã trả `Access-Control-Allow-Origin: *` → dùng `crossorigin="anonymous"` là vẽ được poster.
- Giới hạn: list đầy đủ của user đã ngừng từ 2022 → không có „top 10 bộ bạn xem năm nay“ chính xác; dựa vào tổng thống kê, favorites và lịch sử gần đây.

`/users/{u}/full` `/favorites` `2–3 req / người` `Vừa · 6/10`



Ý TƯỞNG 05 · CHỦ LỰC

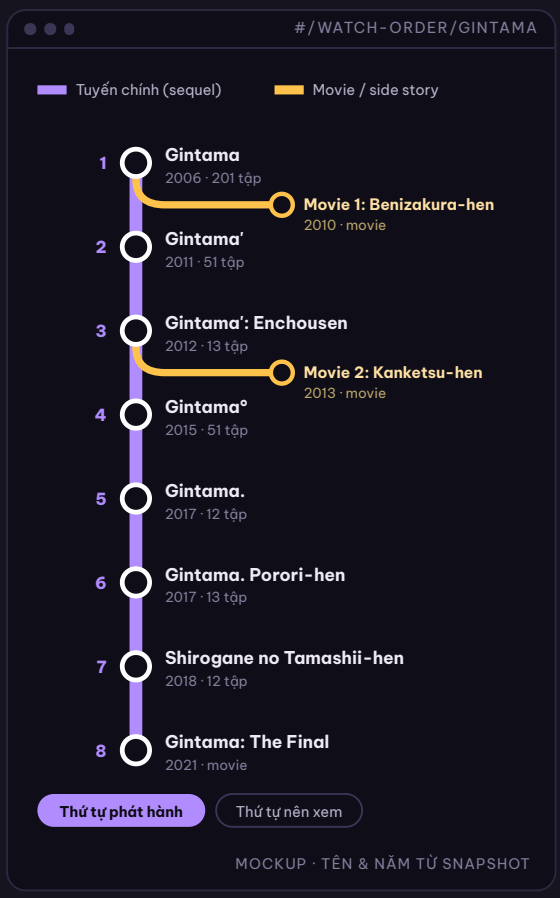
Watch Order — bản đồ tàu điện của franchise

Ồ! Gõ „Gintama“ → cả franchise hiện thành bản đồ metro: tuyến chính là các phần sequel, ga phụ là movie và side story. Chuyển qua lại giữa „thứ tự phát hành“ và „thứ tự nên xem“.

- BFS trên `relations` (có sẵn trong `/full`) từ bộ gốc; phân loại Sequel / Prequel / Side story / Alternative / Spin-off / Summary.
- Sắp theo `aired.from`; vẽ SVG bằng ELK.js hoặc d3-dag theo phong cách metro.
- Tính sẵn ~300 franchise lớn trong pipeline; franchise lạ thì BFS live (giới hạn độ sâu để giữ quota).
- Trả lời câu hỏi phổ biến nhất của người mới: „xem cái nào trước?“.

`/anime/{id}/full → relations` `1 req / mục trong franchise`

Vừa · 6/10



Ý TƯỞNG 06 · CHỦ LỰC

Seiyuu Six Degrees — „cùng một giọng nói“

Ồ! „Frieren và Anya Forger **cùng một giọng** — Tanezaki Atsumi.“ Rồi trò chơi: nối hai nhân vật bất kỳ qua chuỗi lồng tiếng ngắn nhất, kiểu „Six Degrees of Kevin Bacon“.

- Pipeline: `/anime/{id}/characters` cho top 1.500 (≈ 27 phút) → đồ thị hai phía nhân vật ↔ seiyuu (chỉ giữ tiếng Nhật).
- Nén thành danh sách kề; BFS tìm đường ngắn nhất ngay trên trình duyệt (vài ms).
- Trong panel chi tiết: „Giọng này còn lồng cho…“ với ảnh nhân vật — người xem sẽ bấm mãi không dừng.
- Thử thách hằng ngày: „Nối A với B trong ≤ 4 bước“.

`/anime/{id}/characters` `/people/{id}/full` `1.500 req một lần`

Khó · 7/10



Ý TƯỞNG 07 · LÀM NGAY

Score DNA — độ gây tranh cãi

Ồ! Mỗi anime có một „dấu vân tay“ phân phối điểm 1–10. Bảng xếp hạng mà MAL không có: **gây tranh cãi nhất, bị bỏ dở nhiều nhất, khiến người ta xem đến cùng nhất.**

- `/statistics` trả số phiếu từng mức điểm + watching/completed/on_hold/dropped/plan_to_watch.
- **Độ phân cực** = độ lệch chuẩn của phân phối phiếu; **tỉ lệ drop** = dropped / (completed + dropped); **hype gap** = plan_to_watch / total.
- Sparkline histogram nhỏ ngay trên mỗi thẻ anime; trang „Kỷ lục“ với các bảng xếp hạng mới.

`/anime/{id}/statistics` `1 req / anime · Pipeline` `Dễ · 4/10`



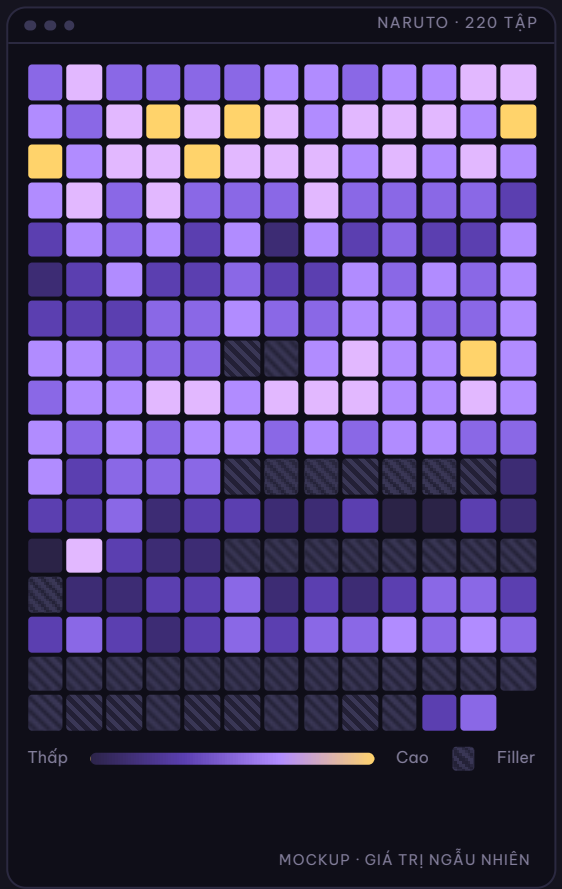
Ý TƯỞNG 08 · CHỦ LỰC

Heatmap tập phim & lộ trình bỏ filler

Ồ! Toàn bộ các tập của một bộ dài hiện thành một lưới màu: nhìn một giây là thấy arc đỉnh cao và những tập nên bỏ. Nút „Lộ trình không filler“ cho Naruto, Bleach, One Piece.

- `/anime/{id}/episodes` (100 tập/trang) có **điểm từng tập 1,00–5,00** do người dùng MAL bình chọn, cùng cờ `filler` và `recap`.
- Heatmap ECharts (đã có trong dự án); ô filler gạch chéo; hover hiện tiêu đề + ngày chiếu.
- Live khi mở + cache 24 h; tính sẵn cho ~200 bộ dài. One Piece (1.100+ tập) ≈ 12 req ≈ 5 s qua hàng đợi → hiển thị dần từng trang.

`/anime/{id}/episodes` `1 req / 100 tập` `Vừa · 5/10`



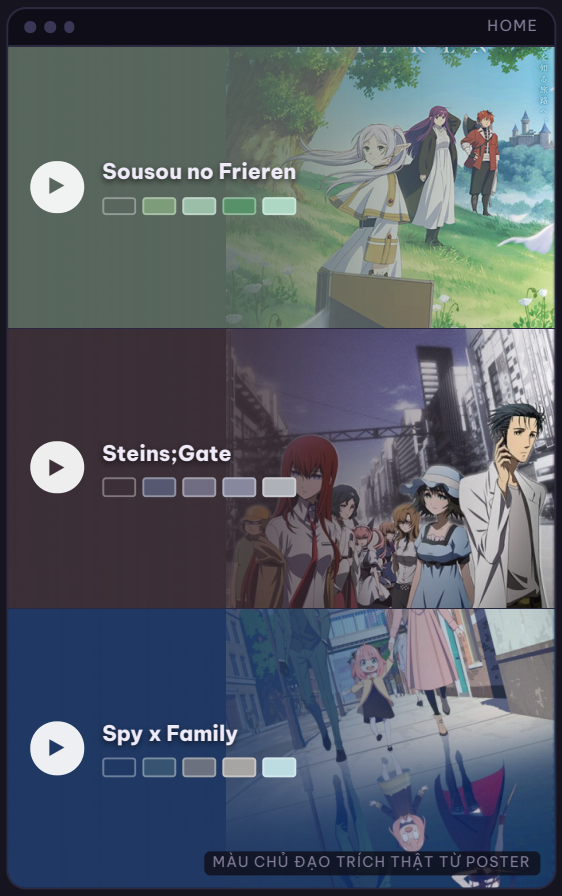
Ý TƯỞNG 09 · LÀM NGAY

Hero điện ảnh + OP Jukebox

Ồ! Trang chủ mở bằng trailer tự chạy (tắt tiếng) của bộ hot nhất mùa; màu của toàn trang đổi theo màu chủ đạo của poster. Kéo xuống là „Jukebox“ các bài OP/ED.

- `trailer.youtube_id` từ `/seasons/now`; dùng facade kiểu *lite-youtube* để chỉ tải player YouTube khi cần.
- Màu chủ đạo tính sẵn trong pipeline (Pillow) và lưu vào index — hoặc tính bằng canvas phía client vì CDN đã mở CORS.
- OP/ED từ `/full → theme.openings/endings`; nút mở tìm kiếm YouTube/Spotify (không cần API).
- Tôn trọng `prefers-reduced-motion`: thay video bằng poster tĩnh.

/seasons/now · /anime/{id}/full · Dễ · 3/10



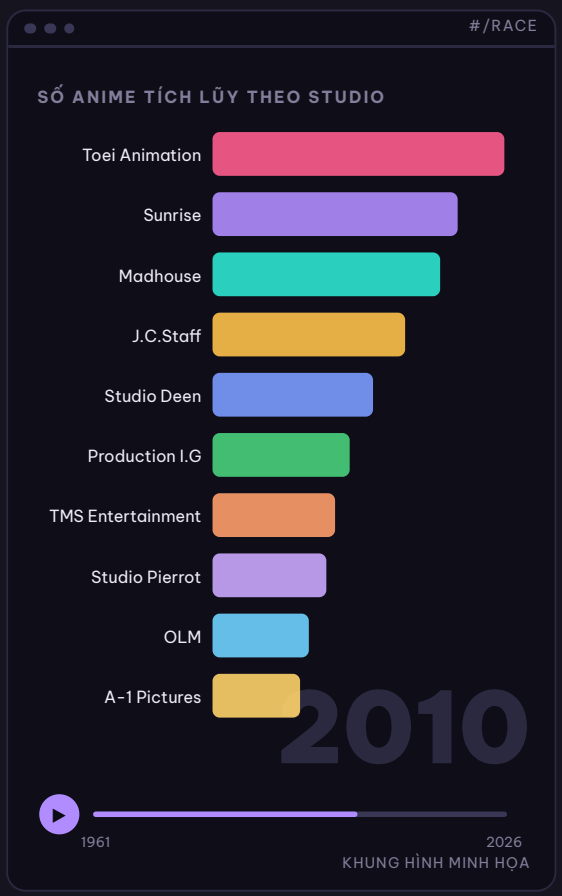
Ý TƯỞNG 10 · LÀM NGAY

Bar chart race — 60 năm anime

Ồ! Một cú bấm Play: các thanh studio/genre đua nhau qua từng năm từ thập niên 60 đến nay. Người xem tự nhìn thấy studio nào trỗi dậy, genre nào lên ngôi.

- ECharts hỗ trợ sẵn (`realtimeSort` + animation) — dự án đã có ECharts.
- **Điều kiện tiên quyết:** sửa lỗi năm (76,6% mục đang thiếu năm), nếu không cuộc đua sẽ sai lệch.
- Biến thể: streamgraph tỉ trọng genre theo thập niên; „Isekai bùng nổ năm nào?“ dựa trên `themes`.
- Xuất video/GIF ngắn bằng MediaRecorder trên canvas để chia sẻ.

0 request · dùng snapshot · Dễ · 2/10



Ý TƯỞNG 11 · NỀN TẢNG

Tracker cá nhân — đúng với cái tên

Ồ! Thêm vào danh sách bằng 1 click, thanh tiến độ tập, huy hiệu mở khóa („Sống sót qua 100 tập“, „Chuyên gia Isekai“), và màn „Năm của tôi“ — không cần tài khoản.

- **Local-first**: IndexedDB (Dexie) lưu trạng thái, tập đã xem, điểm cá nhân, ghi chú.
- Xuất/nhập JSON; chia sẻ cả danh sách qua URL nén (`lz-string`) — không cần server.
- Tùy chọn đồng bộ qua GitHub Gist bằng token của chính người dùng.
- Kết nối các ý tưởng khác: Radar chỉ hiện bộ bạn đang theo; Wrapped nội bộ từ dữ liệu tracker.

0 request · IndexedDB · Vừa · 5/10



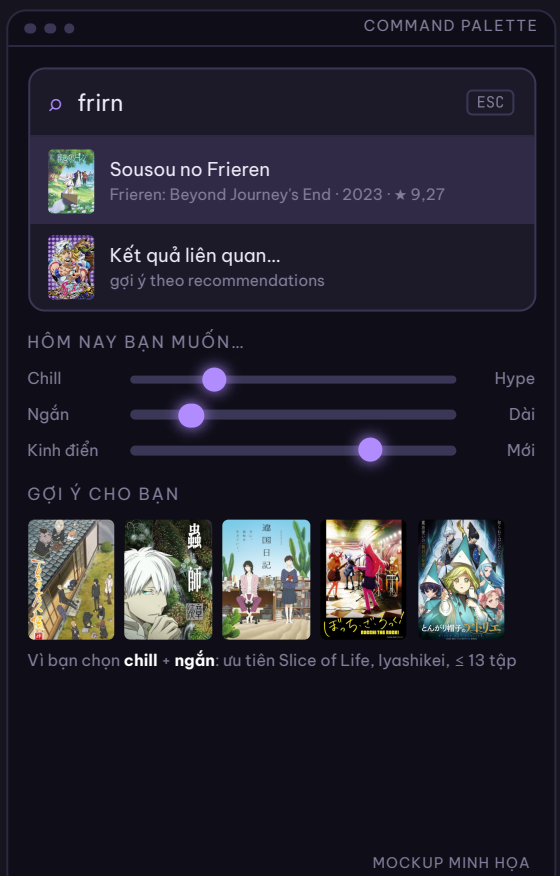
Ý TƯỞNG 12 · LÀM NGAY

⌘K tìm kiếm tức thì + Mood sliders

Ồ! Bấm `/` ở bất cứ đâu: gõ „frirn“ vẫn ra Frieren, tìm cả tên tiếng Anh/Nhật. Mood buttons hiện tại thành 3 thanh trượt → 5 gợi ý kèm lý do „vì bạn chọn…“.

- MiniSearch hoặc Fuse.js chạy trong Web Worker trên index (title, title_english, title_japanese, synonyms).
- Kết quả mở panel chi tiết với deep link `#/anime/52991` — chia sẻ được.
- Mood: chill↔hype, ngắn↔dài, kinh điển↔mới → điểm có trọng số trên genres/themes/episodes/năm/members; giải thích được, không phải hộp đen.

0 request · Web Worker · Dễ · 3/10



04 · CẢM GIÁC „XỊN“

Những chi tiết làm người ta dừng lại

Tính năng làm người ta „Ồ!“ một lần; độ hoàn thiện làm họ ở lại. Danh sách này rẻ nhưng hiệu quả, đa số chỉ là CSS và vài dòng TypeScript.

Nhận diện riêng

- Thay tím Material mặc định (`#bb86fc`) bằng bảng màu riêng: mực đêm + neon sakura + cyan.
- Một font display có cá tính (vd. Be Vietnam Pro, Space Grotesk) + Noto Sans JP cho chữ Nhật trang trí.
- Giữ và nâng cấp Omikuji: mức 大吉/吉/凶 theo điểm, lật thẻ 3D, âm thanh nhẹ, ảnh chia sẻ.

Chuyển động có chủ đích

- View Transitions đã bật → thêm `view-transition-name` để poster „bay“ từ thẻ sang trang chi tiết.
- Thẻ nghiêng 3D theo chuột, viền phát sáng theo màu poster, số KPI đếm chạy.
- Scroll-driven animations (`animation-timeline: view()`) cho trang kể chuyện „Anime qua 60 năm“.

Nhanh là một tính năng

- `loadComponent()` cho mọi route, `@defer (on viewport)` cho phần dưới màn hình.
- Skeleton theo đúng hình thẻ thay cho spinner; `NgOptimizedImage` + webp + kích thước cố định để không nhảy layout.
- `preconnect` tới `cdn.myanimelist.net` và `api.jikan.moe` .

Mọi màn hình, mọi người

- Carousel → CSS scroll-snap; Galaxy/graph hỗ trợ pinch-zoom.
- Chọn một ngôn ngữ UI (Anh cho portfolio quốc tế) + i18n Việt/Pháp; thống nhất ngôn ngữ trong code.
- `prefers-reduced-motion` , tương phản đủ, điều hướng bàn phím, `alt` cho poster.

Chia sẻ được

- Deep link cho mọi thứ: `#/anime/52991` , `#/dle/312` , `#/wrapped/username` .
- Thẻ Open Graph tĩnh đẹp cho site + Web Share API trên mobile.
- PWA (`@angular/service-worker`): cài như app, chạy offline với snapshot.

Minh bạch & ghi công

- „Data: MyAnimeList qua Jikan (không liên kết chính thức với MAL)“ + link về trang MAL của từng anime.
- Nhãn „Cập nhật lúc …“ đọc từ `meta.json` ; ghi rõ khi đang dùng dữ liệu dự phòng.
- README thật: ảnh chụp, GIF demo, kiến trúc, cách chạy pipeline.

Theo mùa & easter egg

- Giao diện đổi theo mùa anime: hoa anh đào rơi mùa xuân, đom đóm mùa hè, lá đỏ mùa thu, tuyết mùa đông (canvas nhẹ, tắt được).
- Konami code → chế độ „Isekai“: mọi poster nghiêng, nhạc 8-bit; gõ tên một anime vào ô Omikuji → bốc trúng 大吉.

Kể chuyện bằng dữ liệu

- Trang cuộn „Anime qua 60 năm“: mỗi thập niên một màn, số liệu thật hiện dần theo cuộn (bar race, streamgraph, poster nổi bật).
- „Hôm nay năm xưa“: anime ra mắt đúng ngày này các năm trước (lọc theo `aired.from` , 0 request).

04 · LỘ TRÌNH

Thứ tự làm để không phải làm lại

Mỗi giai đoạn kết thúc bằng một bản deploy chạy được. Ước lượng cho một người làm bán thời gian.

P0 — ½–1 ngày

Sửa nền

Điểm TB đúng · năm từ `aired` · `sfw=true` · bỏ trùng · gộp `provideRouter` · sửa „Jinkan“ · rank thật. **Kết quả:** mọi con số trên site đều đúng.

P1 — 3–4 ngày

Pipeline & hiệu năng

Crawler Python trong Actions (catalogue tuần + airing ngày) · index 651 KB + mảnh synopsis + `meta.json` · `DataStore` + Web Worker · lazy route · ECharts modular · chuyển `backend/` thành `pipeline/`. **Kết quả:** tải lần đầu giảm ~87%, dữ liệu tự cập nhật.

P2 — ~1 tuần

Lớp sống

`JikanClient` (hàng đợi + cache + retry) · trang/panel chi tiết `#/anime/:id` · Radar phát sóng · ⌘K. **Kết quả:** site có lý do để mở mỗi tuần.

P3 — 1–2 tuần

Lan truyền

Animedle · Hero điện ảnh + Jukebox · Bar chart race · Score DNA · ảnh chia sẻ. **Kết quả:** thứ để người xem gửi cho bạn bè.

P4 — 2–3 tuần

Chủ lực

Anime Galaxy (thay graph) · Watch Order · Seiyuu Six Degrees · Heatmap tập · MAL Wrapped · Tracker. **Kết quả:** trải nghiệm không site anime nào khác có đủ.

ĐO THÀNH CÔNG BẰNG GÌ

< 300 KB	JS ban đầu (gzip) — hiện 558 KB
< 700 KB	dữ liệu ban đầu (gzip) — hiện 5,05 MB
< 2,5 s	LCP trên mạng 4G
≤ 24 h	tuổi tối đa của dữ liệu airing (catalogue ≤ 7 ngày)

RỦI RO & CÁCH GIẢM

Jikan sập / chậm (đã gặp khi viết tài liệu này)	Snapshot là nguồn chính, live chỉ là lớp làm mới; pipeline thất bại thì giữ dữ liệu cũ, không commit file rỗng.
MAL chặn Jikan → 429 hàng loạt	Lùi lũy thừa, giảm nhịp còn 30/phút khi gặp 429 liên tiếp, tiếp tục ở lần chạy sau (lưu vị trí trang).
Cron bị GitHub tắt sau 60 ngày	Workflow tự commit dữ liệu; thêm `workflow_dispatch` để chạy tay; nhãn „Cập nhật lúc…“ giúp phát hiện sớm.
Bundle phình ra vì WebGL, chart, game	Mỗi tính năng là một route lazy; thư viện nặng chỉ tải khi vào trang đó; budget trong `angular.json`.
Bản quyền & điều khoản	Ghi công MAL/Jikan, link về trang gốc, không tự host lại poster số lượng lớn, tuân thủ điều khoản sử dụng của Jikan.

PHỤ LỤC A

JikanClient: tôn trọng limit phía trình duyệt

Một service duy nhất cho cả app: giãn nhịp, trần theo phút, gộp request trùng, cache IndexedDB 24 h, retry lũy thừa và rơi về dữ liệu cũ. Dùng `idb-keyval` cho phần lưu trữ.

```ts
import { Injectable } from '@angular/core';
import { get as idbGet, set as idbSet } from 'idb-keyval';

const sleep = (ms: number) => new Promise(r => setTimeout(r, ms));
type Entry<T> = { t: number; v: T };

@Injectable({ providedIn: 'root' })
export class JikanClient {
  private readonly BASE = 'https://api.jikan.moe/v4';
  private readonly GAP_MS = 400;        // ≤ 2,5 req/s   (trần Jikan: 3/s)
  private readonly PER_MIN = 50;        // chừa 10 lượt   (trần Jikan: 60/phút)
  private readonly TTL = 24 * 3600_000;  // khớp cache 24 h của Jikan
  private sent: number[] = [];
  private gate: Promise<void> = Promise.resolve();
  private inflight = new Map<string, Promise<unknown>>();

  /** Ví dụ: client.get<AnimeFull>('/anime/52991/full') */
  get<T>(path: string): Promise<T> {
    const pending = this.inflight.get(path);
    if (pending) return pending as Promise<T>;                // gộp request trùng
    const p = (async () => {
      const hit = await idbGet<Entry<T>>(path);
      if (hit && Date.now() - hit.t < this.TTL) return hit.v;  // cache tươi: 0 request
      return this.fetchWithRetry<T>(path, hit?.v);
    })().finally(() => this.inflight.delete(path));
    this.inflight.set(path, p);
    return p;
  }

  private async fetchWithRetry<T>(path: string, stale?: T): Promise<T> {
    for (let attempt = 0; attempt < 4; attempt++) {
      await this.slot();                                      // chờ tới lượt
      try {
        const res = await fetch(this.BASE + path, { signal: AbortSignal.timeout(10_000) });
        if (res.ok) {
          const v = (await res.json()) as T;
          await idbSet(path, { t: Date.now(), v } satisfies Entry<T>);
          return v;
        }
        if (res.status !== 429 && res.status < 500) break;         // 400/404: thử lại vô ích
      } catch { /* timeout / mạng lỗi → thử lại */ }
      await sleep(1000 * 2 ** attempt + Math.random() * 300);     // 1, 2, 4, 8 s + jitter
    }
    if (stale !== undefined) return stale;                         // dữ liệu cũ vẫn hơn màn hình lỗi
    throw new Error(`Jikan không phản hồi: ${path}`);
  }

  /** Hàng đợi tuần tự: cách nhau ≥ GAP_MS và ≤ PER_MIN request trong 60 s gần nhất. */
  private slot(): Promise<void> {
    const turn = this.gate.then(async () => {
      const now = Date.now();
      this.sent = this.sent.filter(t => now - t < 60_000);
      let wait = Math.max(0, (this.sent.at(-1) ?? 0) + this.GAP_MS - now);
      if (this.sent.length >= this.PER_MIN) wait = Math.max(wait, this.sent[0] + 60_000 - now);
      await sleep(wait);
      this.sent.push(Date.now());
    });
    this.gate = turn;
    return turn;
  }
}
```

PHỤ LỤC B

Pipeline: GitHub Actions + crawler

Một workflow làm cả ba việc: crawl → commit dữ liệu → build & deploy. Crawler giữ nhịp 1,1 s, retry có kiểm soát và lấy năm từ `aired`.

```yaml
# .github/workflows/data.yml
name: refresh-data
on:
  schedule:
    - cron: '17 3 * * *'  # 03:17 UTC, tránh phút :00
  workflow_dispatch:
permissions:
  contents: write
  pages: write
  id-token: write
concurrency: data
jobs:
  crawl-build-deploy:
    runs-on: ubuntu-latest
    timeout-minutes: 120
    environment: github-pages
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.12' }
      - run: pip install requests
      - run: python pipeline/crawl.py --airing
             --enrich-shard $(( $(date +%u) % 7 ))
      - name: Commit dữ liệu
        run: |
          u="github-actions[bot]"; d="users.noreply.github.com"
          git config user.name "$u"
          git config user.email "41898282+$u@$d"
          git add front/public/data
          git diff --cached --quiet ||
            git commit -m "data: $(date -u +%F)"
          git push
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
          cache-dependency-path: front/package-lock.json
      - run: npm ci && npx ng build
             --base-href /AnimeTracking/
        working-directory: front
      - uses: actions/upload-pages-artifact@v3
        with:
          path: front/dist/animetracking-ui/browser
      - uses: actions/deploy-pages@v4

# Push bằng GITHUB_TOKEN không kích hoạt workflow
# khác → build + deploy ngay trong job này.
# Bật Settings → Pages → Source: GitHub Actions.
# Kiểm tra lại phiên bản mới nhất của các action.
```

```python
# pipeline/crawl.py (rút gọn)
import random, time, requests

BASE = "https://api.jikan.moe/v4"
GAP = 1.1          # ≈ 55 req/phút < trần 60
RETRY = {429, 500, 502, 503, 504}
S = requests.Session()
S.headers["User-Agent"] = ("AnimeTracking-pipeline "
    "(github.com/TriNGUYEN0/AnimeTracking)")
_last = 0.0

def get(path, params=None, etag=None, tries=5):
    global _last
    for i in range(tries):
        time.sleep(max(0, _last + GAP - time.time()))
        _last = time.time()
        hdr = {"If-None-Match": etag} if etag else {}
        try:
            r = S.get(BASE + path, params=params,
                      headers=hdr, timeout=30)
        except requests.RequestException:
            r = None
        if r is not None:
            if r.status_code == 304:
                return None, etag   # không đổi
            if r.ok:
                return r.json(), r.headers.get("ETag")
            if r.status_code not in RETRY:
                r.raise_for_status()
        time.sleep(2 ** i + random.random())  # 1…16 s
    raise RuntimeError(f"bỏ cuộc: {path}")

def year_of(a):
    if a.get("year"):
        return a["year"]
    prop = (a.get("aired") or {}).get("prop") or {}
    return (prop.get("from") or {}).get("year")

def slim(a):
    return {
        "id": a["mal_id"], "t": a["title"],
        "y": year_of(a), "s": a["score"] or None,
        "m": a["members"], "r": a["rank"],
        "g": [x["name"] for x in a["genres"]],
        "th": [x["name"] for x in a["themes"]],
        "yt": (a["trailer"] or {}).get("youtube_id"),
    }

def catalogue():
    out, page = {}, 1
    while True:
        js, _ = get("/anime", {"page": page, "limit": 25,
                    "sfw": "true", "order_by": "mal_id"})
        for a in js["data"]:
            out[a["mal_id"]] = slim(a)  # dict → bỏ trùng
        if not js["pagination"]["has_next_page"]:
            return out
        page += 1
```

NGUỒN

Nguồn & cách đo

TÀI LIỆU

- docs.api.jikan.moe — Jikan REST API v4: rate limit (3/s, 60/phút, không giới hạn/ngày), cache 24 h, ETag/304, mã lỗi, tham số.
- github.com/jikan-me/jikan-rest · `storage/api-docs/api-docs.json` — OpenAPI spec: 100 endpoint, schema episodes (score 1–5, filler, recap), statistics, characters, broadcast; animelist ngừng từ 01/05/2022.
- github.com/jikan-me/jikan-rest/issues/461 — maintainer: trần là 60 request/phút.
- docs.github.com/en/pages/getting-started-with-github-pages/github-pages-limits — 1 GB, 100 GB/tháng, timeout 10 phút, 10 build/giờ.
- docs.github.com/en/actions/how-tos/manage-workflow-runs/disable-and-enable-workflows — workflow schedule tự tắt sau 60 ngày không hoạt động (repo public).

ĐO ĐẠC TRONG TÀI LIỆU NÀY (09/11/2026)

- Mọi số liệu dữ liệu đo bằng Python trên `front/public/assets/animes_graphe.json` (commit `eca2665`).
- Index tối ưu: lọc Hentai/Erotica, bỏ trùng, mảng + bảng tra, nén gzip mức 9.
- Kích thước live (gzip 5,16 MB, main.js 1,7 MB / 558 KB gzip, `max-age=600`): `curl` tới tringuyen0.github.io.
- CDN ảnh MAL: `Access-Control-Allow-Origin: *`; poster Frieren .jpg 56.653 B vs .webp 15.746 B.
- `api.jikan.moe`: timeout kết nối liên tục khi đo (trong khi jikan.moe và MAL vẫn trả 200).
- Màu chủ đạo ở ý tưởng 09: lượng tử hóa median-cut 6 màu trên poster thật.

> Mockup trong tài liệu là minh họa ý tưởng; chỗ nào dùng số liệu thật thì đã ghi rõ. Poster và dữ liệu anime thuộc MyAnimeList và các bên giữ bản quyền.

Checklist P0 — bắt đầu ngay hôm nay

- ☐ `dashboard.ts:114` & `analytics.service.ts:57`: chia cho số anime **có điểm**
- ☐ Crawl lại với `sfw=true`, bỏ `rating=rx`
- ☐ `app.config.ts:11-13`: gộp một `provideRouter(…)`
- ☐ `home.html:42`: ngày cập nhật đọc từ dữ liệu
- ☐ URL ảnh → `cdn.myanimelist.net/…webp`
- ☐ `extraire_graphe.py:23`: năm từ `aired.prop.from.year`
- ☐ Bỏ trùng theo `id` (148 mục)
- ☐ `home.html:40`: „Jinkan“ → „Jikan“
- ☐ Một `DataStore` thay cho 4 lần `http.get`
- ☐ Ghi công „Data: MyAnimeList via Jikan“ ở footer